

Big Data for Electronic Design Automation: Probabilistic Sketches and GPU Acceleration for Chip Analysis

Wai-Shing Luk

Abstract—Modern integrated-circuit design flows emit event streams — logic simulation toggles, design-rule violations, placement and timing objects — whose volume grows with the design itself, while the questions the tools ask of those streams (how often did this net toggle, how many distinct patterns occurred, which nodes are hot spots) are aggregate answers that do not require the raw events to be stored. This paper studies single-pass probabilistic summaries — the Count-Min Sketch for frequencies and HyperLogLog for cardinalities — as the measurement layer of electronic design automation (EDA) analytics. We instantiate both structures with deterministic, mergeable hashing, derive their sizing rules and error guarantees, and evaluate them on a suite of EDA counting problems: toggle counts, unique design-rule and defect patterns, activity signatures, and coverage state spaces. A switching-power case study shows that a 53.1KB sketch summarizes 64,106 toggle events from a 500-node design and recovers category-level power and hot spots, and a memory-accuracy sweep quantifies exactly when a sketch breaks: a 39KB summary keeps hot-node error at 5.8% while the tail collapses. Across twelve EDA metrics the HyperLogLog estimates stay within 3.4% of ground truth in a single pass with no stored elements. We further describe race-free GPU kernels driven by hardware atomics, and argue that sketches are the memory-bounded sensing layer that makes real-time, AI-driven design loops affordable. All experiments are reproducible from the accompanying open-source package.

I. INTRODUCTION

A modern system-on-chip design has millions of gates and hundreds of thousands of nets, and a single logic simulation may sweep 10^9 cycles against that netlist [1]. Every node toggle is one event in a stream, so exact bookkeeping of per-net activity, per-pattern counts, or per-state coverage costs $O(n)$ storage in the number of distinct keys — storage that grows with the design and must be maintained pass after pass. Yet the questions EDA tools and designers actually ask are aggregates: which nets are the hot spots, how many distinct violation signatures appeared, did coverage close. The raw stream is unbounded; the answer is small.

The streaming-database community answered this mismatch with probabilistic *sketches*: summaries of sub-linear size that answer aggregate queries with a bounded, quantifiable error in a single pass over the data [2], [3], [4]. Sketches proved their worth at line rate in network measurement [5], where packets arrive too fast to store.

Toggle-event streams in simulation and analysis share the same structure, and — like network telemetry — they can be consumed, merged across runs or chiplets, and then discarded.

This paper brings two workhorse sketches to EDA, works out the sizing and error rules that make them trustworthy for design decisions, and validates them on counting problems drawn from real analysis flows: switching-activity tracking, design-rule violation signatures, temporal activity windows, pattern-dependent and multi-corner sensitivity, and verification coverage.

Our contributions are as follows.

1. We formulate representative EDA counting problems — toggle counts, unique pattern counts, coverage cardinalities — as stream estimation problems, and instantiate the Count-Min Sketch with deterministic universal hashing derived from its dimensions, which makes equal-sized sketches share hash functions and therefore merge correctly (Section IV).
2. We adapt HyperLogLog with its standard bias corrections to EDA objects via typed signatures, and show that a single cardinality primitive supports seven distinct estimators — from unique design-rule patterns to failing-die buckets — all with $m = 2^p$ registers (Section V).
3. We present a switching-power case study in which a 53.1KB sketch summarizes 64,106 toggle events in under a second and recovers per-category power, temporal burstiness, the current envelope, and pattern/multi-corner sensitivity (Section VII).
4. We quantify the memory-accuracy trade-off empirically on a 10,000-node power-law workload, showing that hot-node *ranking* is robust at medium sketch sizes while the distribution tail is not (Section VII).
5. We describe race-free GPU kernels for HyperLogLog in which one thread per hash applies hardware atomic maximum to shared registers, and we report accuracy across precisions and batch sizes (Section VI).

II. RELATED WORK

Estimating frequencies in a stream has a long history: Misra–Gries heavy-hitters [6], lossy counting [7], and the F_0 – F_2 moment sketches of Alon, Matias and Szegedy [8] establish that sub-linear space with probabilistic guarantees

is possible. The Count-Min Sketch [2] dominates practice because its update is a handful of increments and its one-sided error — collisions only ever over-count — is easy to reason about; it is the structure we use for toggle and activity frequencies. Cormode et al. survey the wider synopsis family (samples, histograms, quantiles) in [4].

For cardinality, linear counting [9] counts distinct keys from a bitmap of hashed positions; LogLog [10] observed that the maximum run length of leading zeros in uniform hashes is an unbiased estimator of $\log_2 n$; and HyperLogLog [3] replaced the mean with the harmonic mean over registers to cut the standard error to $1.04/\sqrt{m}$ while keeping $O(m)$ state, at the price of two bias corrections for the small and large ranges. Heule et al. document the engineering of a production-grade implementation in [11]. Our implementation follows the classical formulation, including both corrections.

In networking, sketches are standard equipment: Estan and Varghese [5] introduced bitmap and hash-based counters precisely because flow measurement could not afford per-flow state. EDA analytics face an analogous wall — simulation produces toggle streams at a rate no exact table sustains — but the EDA literature has comparatively few studies that treat analysis outputs as streams with formal error budgets. This paper closes that gap for a representative family of EDA counting problems, and connects the summaries to the GPU so that the hashing and register updates run at accelerator rates.

III. PRELIMINARIES

A. The streaming model

An analysis produces a sequence of events $x_1, \dots, x_N$ (toggles, violations, pattern identifiers, coverage states). A sketch is a fixed-size summary S updated once per event and queried afterwards; the events are not stored, and two summaries of the same dimensions can be merged element-wise. The design question is therefore a three-way trade between memory, error, and the cost of exactness, summarized in Table~I.

Both structures below satisfy a guarantee of the form: with probability at least $1 - \delta$,

$$\Pr[\hat{c}_x \leq c_x + \varepsilon N] \geq 1 - \delta, \quad (1)$$

where c_x is the true count of key x , $\hat{c}_x$ the estimate, and N the number of events — an absolute additive error of εN that is *independent of the key's popularity*, which is exactly the property hot-spot ranking needs.

IV. COUNT-MIN SKETCH

A. Structure and update

A Count-Min Sketch is a $d \times w$ array C of counters with d independent hash functions. Keys are integers (node identifiers or hashes of node names). We use a universal hash over the Mersenne prime $p = 2^{31} - 1$,

$$h_i(x) = ((a_i x + b_i) \bmod p) \bmod w, \quad (2)$$

with parameters (a_i, b_i) drawn from a generator seeded by the sketch dimensions — a choice with two consequences we rely on: every run of the experiment computes identical estimates, and two sketches of equal dimensions hash identically, so their merge is correct.

Updating a key increments one counter in every row,

$$\forall i \in [d] : C[i, h_i(x)] += 1, \quad (3)$$

and a query returns the minimum over the rows,

$$\hat{c}_x = \min_{i \in [d]} C[i, h_i(x)]. \quad (4)$$

Because a collision can only add foreign counts, every row is an over-estimate, and the minimum of over-estimates is the tightest safe one: the estimate never under-counts.

B. Sizing

For target additive error εN with confidence $1 - \delta$, the standard dimensions are

$$w = \lceil e/\varepsilon \rceil, \quad d = \lceil \ln(1/\delta) \rceil. \quad (5)$$

For $\varepsilon = 0.001$ and $\delta = 0.01$ this gives $w = 2719$, $d = 5$, i.e. $2719 \times 5 \times 4 \text{B} \approx 53.1 \text{KB}$ of counters for an error budget of $\pm 10^5$ counts at $N = 10^8$ events. The additive budget is absolute, so error *relative* to a rare key can be large even though the guarantee holds — the reason tail queries must be sized from the query, not the stream (Section VII).

C. Switching-activity tracking

The smallest demonstration is instructive: a $w = 50$, $d = 3$ sketch (600 counters) tracking six nets over 235 toggle events returns the exact counts of the three active nets with no collision (Table~II), so hot spots are visible after a single pass and nothing is stored but the array.

V. HYPERLOGLOG

A. Structure and estimate

HyperLogLog estimates the number of *distinct* keys — the primitive behind every “how many” question that counts objects rather than occurrences: unique design-rule signatures, distinct failing-die buckets, coverage states. The data structure is an array of $m = 2^p$ registers, one per hash range. Each key is hashed to 64 bits; the register index uses the top p bits and the counter uses the leading zeros of the remainder,

$$j = x \gg (64 - p), \quad \rho(x) = 1 + \text{clz}(x), \quad (6)$$

and each register keeps the maximum rank it has seen,

$$M[j] \leftarrow \max(M[j], \rho). \quad (7)$$

Table I: Exact counting versus a sketch summary. The sketch spends a probabilistic, quantitatively bounded error to remove the $O(n)$ storage and the need for a second pass.

	exact counting	sketch
memory	$O(n)$ in distinct keys	$O(\epsilon^{-1} \log \delta^{-1})$ counters
query	exact	probabilistic bound
one-pass stream	requires storage	yes
merge across runs	merge tables	element-wise add

Table II: Count-Min Sketch estimates for six nets over 235 toggle events ($w = 50$, $d = 3$). The three active nets are recovered exactly at this size.

net	node ID	true toggles	estimate
clk_buffer_1	1001	150	150.0
data_reg_1	2001	75	75.0
memory_cell	4001	10	10.0
others	—	0	0.0

If all hashes were uniform, the expected maximum rank in a register would encode $\log_2 n$; summing over registers with the harmonic mean gives the cardinality estimate

$$\hat{n} = \frac{\alpha_m m^2}{\sum_{j=1}^m 2^{-M[j]}}, \quad \alpha_m = \begin{cases} 0.673 & m = 16 \\ 0.697 & m = 32 \\ 0.709 & m = 64 \\ \frac{0.7213}{1 + 1.079/m} & m \geq 128 \end{cases} \quad (8)$$

where α_m corrects the small-range bias of the harmonic estimator. Two further corrections handle the ends of the range: for small $\hat{n}$, when the raw estimate is below $2.5m$ and there are empty registers, a linear counting estimate $\hat{n} = m \ln(m/V)$ (with V empty registers) is used instead [9];

$$\hat{n} = m \ln(m/V) \quad \text{if } \hat{n} \leq 2.5m \text{ and } V > 0, \quad (9)$$

and for the large range the raw estimate is biased by floating-point effects, corrected with the classical alternative $H = -\sum 2^{-M[j]}$,

$$\hat{n} = -2^{64} \ln(1 - \hat{n}/2^{64}) \quad \text{if } \hat{n} \leq 2^{64}/30. \quad (10)$$

For EDA objects, the hashed key is a *typed signature* — the violation class, node identifier, or defect bucket combined with its domain tag — so that distinct object families hash into distinct streams and one primitive serves many queries.

B. Error and memory

The standard error of HyperLogLog is

$$\sigma(\hat{n}) \approx \frac{1.04}{\sqrt{m}}, \quad (11)$$

independent of n : precision $p = 14$ ($m = 16,384$ registers, 64KB at the classical 4 bytes per register, or 12KB with the 6-bit packing used in our implementation) gives about 0.81% one-sigma error; each additional bit of precision reduces the error by $\sqrt{2}$ and doubles the memory. Unlike exact sets, the memory is *fixed* and known before the run — the property that lets a chip-design tool budget its analysis state.

Table~III lists the design points; Table~IV compares them against measured relative error on EDA counting workloads. The measured errors track the $1.04/\sqrt{m}$ curve, and the packed implementation reaches 12KB at $p = 14$ — two orders of magnitude below the exact set it summarizes.

The merge test is worth noting: merging two sketches of 5,000 distinct keys each yields 10,457 against a true union of 10,000 — a 4.6% over-estimate. Because registers hold maxima, a merge is associative, commutative, and exact in the same sense the single-pass estimate is: parallel runs over chip partitions summarize and combine without a coordinating reduction of the raw events.

VI. GPU-ACCELERATED SKETCHES

Sketches are attractive on GPUs for the same reason they are attractive on streams: updates are tiny, independent, and numerous. Our HyperLogLog kernel follows the data-parallel formulation of Mirhoseini et al. [12] for hardware-accelerated placement, adapted to counting:

- **Hashing.** Each thread hashes one 64-bit item with MurmurHash3's `fmix64` finalizer (constants `0xFF51AFD7ED558CCD` and `0xC4CEB9FE1A85EC53`), then extracts the register index from the top p bits and the rank from the leading-zero count of the low bits.
- **Update.** The register update is a maximum, issued as a hardware `atomicMax` on shared device memory — race-free without locks, because concurrent threads may only raise a register, never lower it.
- **Reduction.** Registers are summed in log steps; the harmonic-mean accumulation follows the register pass with double precision to avoid cancellation at large cardinalities.

Table~V lists the kernel design points: the bucket count m determines both memory and the error envelope, exactly as in Section V. The batch-size sweep in `hyper_loglog_gpu_demo` (5M items, 500k unique, batches from 10^3 to 10^6 per launch) confirms that launch granularity does not affect the estimate: the final register state depends only on the set of hashed items, so throughput tuning — the lever that determines kernel efficiency — costs

Table III: HyperLogLog design points: standard error $1.04/\sqrt{m}$ versus memory — classical 4 bytes per register versus the 6-bit packing of our implementation — with measured relative error on EDA counting workloads.

precision p	registers m	theory error	measured error	$m \times 4\text{ B}$	measured ($m \times 6/8$)
8	256	6.50%	0.98%	1 KB	0.2 KB
10	1,024	3.25%	2.20%	4 KB	0.8 KB
12	4,096	1.63%	2.73%	16 KB	3.0 KB
14	16,384	0.81%	0.46%	64 KB	12 KB
16	65,536	0.41%	0.02%	256 KB	48 KB

Table IV: Measured relative error of HyperLogLog against ground truth on EDA counting workloads (v0.2.0 run, $p = 14$ unless noted). Errors fall with m and stay inside the predicted envelope at practical sizes.

workload	estimate	ground truth	absolute error	relative error
design-rule patterns	300,877	290,919	9,958	3.4%
signal paths	498,998	500,000	1,002	0.2%
timing objects	99,503	100,000	497	0.5%
critical paths	49,856	50,584	728	1.4%
merge test ($2 \times 5,000$)	10,457	10,000	457	4.6%

no accuracy. The demo reports `cpu_time / gpu_time` at runtime, so the observed speed-up is measured on whatever GPU the experiment runs on rather than asserted a priori.

The same pattern — fixed memory, quantified error, merges — carries over to the Count-Min Sketch on the GPU, where increments become `atomicAdd`. The design implication is that the analysis layer can run entirely in device memory: a full-chip activity summary fits in tens of kilobytes, well inside the on-chip budget of any modern accelerator.

VII. CASE STUDY: SWITCHING POWER ANALYSIS

A. From toggles to power

We now exercise both sketches on a complete analysis flow: a switching-power pipeline over a 500-node design simulated for 1000 cycles, producing 64,106 toggle events. The full-chip summary is produced in under a second and the sketch state is 53.1,KB — two orders of magnitude below the raw event log.

Activity factors estimated from the sketch feed the classical dynamic-power formula,

$$P = \alpha C V_{\text{dd}}^2 f, \quad (12)$$

with α the toggle activity of each category, C its total capacitance, V_{dd} the supply, and f the clock frequency. With per-node toggle counts from the Count-Min Sketch, a temporal analyzer segments the event stream into windows and computes the burstiness index

$$B = \frac{\sigma - \mu}{\sigma + \mu}, \quad \mu, \sigma \text{ of the windowed activity,} \quad (13)$$

and the peak-to-average current envelope via $I_{\text{peak}} = CV/\Delta t$ with $\Delta t = 50\text{ ps}$. A pattern score summarizes deviation across analysis patterns, and the sensitivity index (SI) reports the spread of activity across them.

B. Power breakdown and temporal behavior

Table~VI gives the per-category power. Clock distribution dominates: 50 clock nodes with 5,fF load consume $125.0\text{ }\mu\text{W}$ of the $194.3\text{ }\mu\text{W}$ total — 64% — while the 200 memory

cells with 1,fF loads contribute $5.9\text{ }\mu\text{W}$. This is the classic result that clock networks dominate dynamic power, here derived entirely from sketch estimates rather than a stored event table.

Figure~1 shows the full breakdown produced by the pipeline: per-category activity factors feeding the power formula, the resulting per-category and total power, and the hot-node ranking extracted from the same sketch state.

Segmenting a 100-cycle window with a $w = 500$, $d = 4$ sketch, the burstiness index separates four behavioral regimes (Table~VII): a periodic clock (B rising $0.00 \rightarrow 0.50$ as windows empty between edges), bursty activity ($0.35 \rightarrow 0.16$), random traffic ($0.30 \rightarrow 0.17$, nearly flat because randomness is self-similar across windows), and a quiet circuit ($0.50 \rightarrow 0.02$). The current envelope over the same events peaks at 0.34 mA against a 0.04 mA average — an $8.5\times$ peak-to-average ratio that a mean-only analysis would have missed (Figure~2).

C. Patterns and corners

Repeating the analysis per pattern and per process corner keeps the sketch state fixed while multiplying the questions asked of it. Across four patterns the violation-style activity counts are Idle 1,286, Reset 2,250, Max_Load 3,121, Worst_Case 3,997, with sensitivity index peaking at $\text{SI} = 4.0$ for Max_Load — the pattern in which the largest share of nodes switch together. Across corners, TT/25°C draws $53.0\text{ }\mu\text{W}$, FF/125°C $72.6\text{ }\mu\text{W}$, SS/−40°C $28.5\text{ }\mu\text{W}$, and the turbo corner $49.5\text{ }\mu\text{W}$: an 86.6% spread between best and worst corner, worst at fast-hot — again all from fixed-size summaries per corner that merge alongside the others (Figure~3).

D. When the sketch breaks: the memory-accuracy frontier

The last experiment is the honest one. A 10,000-node power-law design generates 119,266 events; a Count-Min Sketch of increasing size estimates the power of every node, and both global and hot-node errors are measured against exact counts (Table~VIII).

Table V: GPU HyperLogLog kernel design points. Memory is always $m \times 4$ bytes; 64 KB of registers holds $m = 16,384$ counters at $p = 14$.

kernel	buckets	precision	target error
$b = 14$	16,384	$p = 14$	$\approx 1\%$
$b = 16$	65,536	$p = 16$	$\approx 0.4\%$

Table VI: Per-category dynamic power of the 500-node design, from Count-Min-Sketch activity factors (V_{dd} and f fixed across categories). Total $\approx 194.3 \mu\text{W}$, of which the clock network is 64%.

category	nodes	load per node	activity	power
clock	50	5 fF	0.50	125.0 μW
logic	150	2 fF	0.20	60.5 μW
control	100	1 fF	0.15	2.9 μW
memory	200	1 fF	0.15	5.9 μW
total	500	—	—	$\approx 194.3 \mu\text{W}$

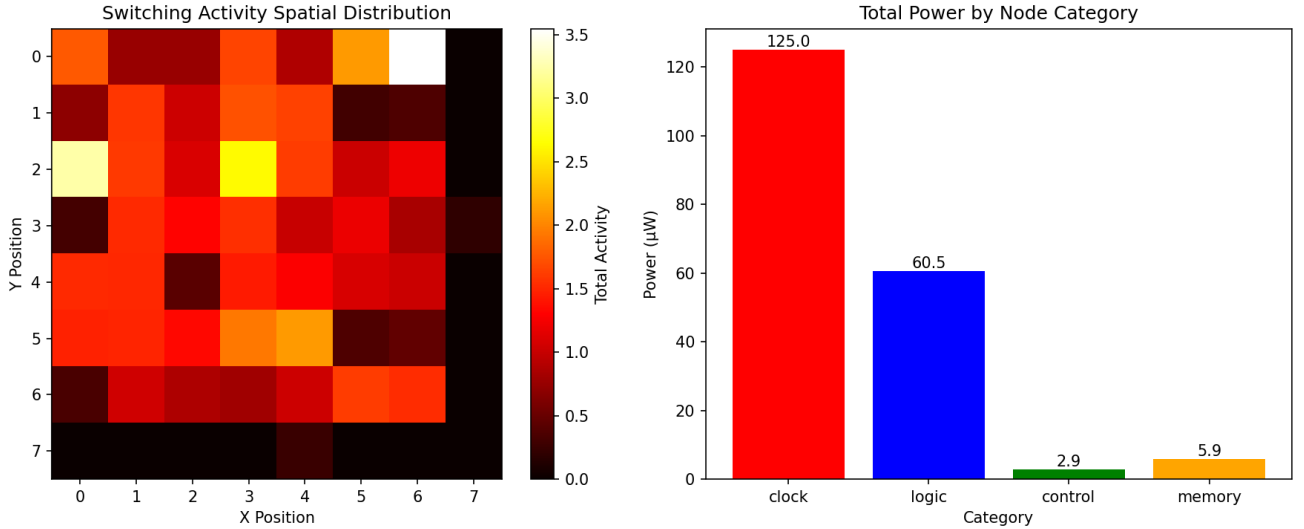


Figure 1: Switching-power analysis of the 500-node design: activity factors, per-category power, and hot-node ranking, all derived from sketch estimates with no stored event log.

Table VII: Burstiness index B over 100-cycle windows ($w = 500$, $d = 4$) for four behavioral regimes. B separates periodic from random activity without storing the event stream.

regime	early window	late window
periodic clock	0.00	0.50
bursty	0.35	0.16
random	0.30	0.17
quiet	0.50	0.02

Three regimes emerge. Below ~ 10 KB the sketch is dominated by collisions: mean error is catastrophic and even the hottest node is misestimated by 323%. At 39KB the hot-node error has already fallen to 5.8% — enough to *rank* the dominant switching nodes and draw floorplan or clock-tree conclusions — while the distribution tail is still off by 349% on average. At 234KB both are essentially exact. The practical rule for EDA is therefore query-driven: size the sketch from the *smallest count you intend to act on*, because the additive guarantee Eq. 1 is absolute while the query threshold is what defines usable accuracy (Figure~4).

VIII. ESTIMATES VERSUS GROUND TRUTH

Table~IX consolidates the twelve-metric EDA suite. Every row is a single-pass estimate against an exact count, with

the sketch state measured rather than derived: the DRC-pattern metric sits at the 3.4% worst case, most metrics are below 1%, and several (fanout degrees, transition counts, covered states) are exact. All rows use the same $p = 14$ precision (12KB measured) except where noted.

The pattern to notice is that error is *uniform in principle* ($1.04/\sqrt{m}$ is independent of the true count) but *unequal in practice*: heavy keys collide less often in relative terms, so the metrics whose values are large (498,998 signal paths) come out closest to ground truth while the smaller cardinalities absorb proportionally more of the same absolute noise.

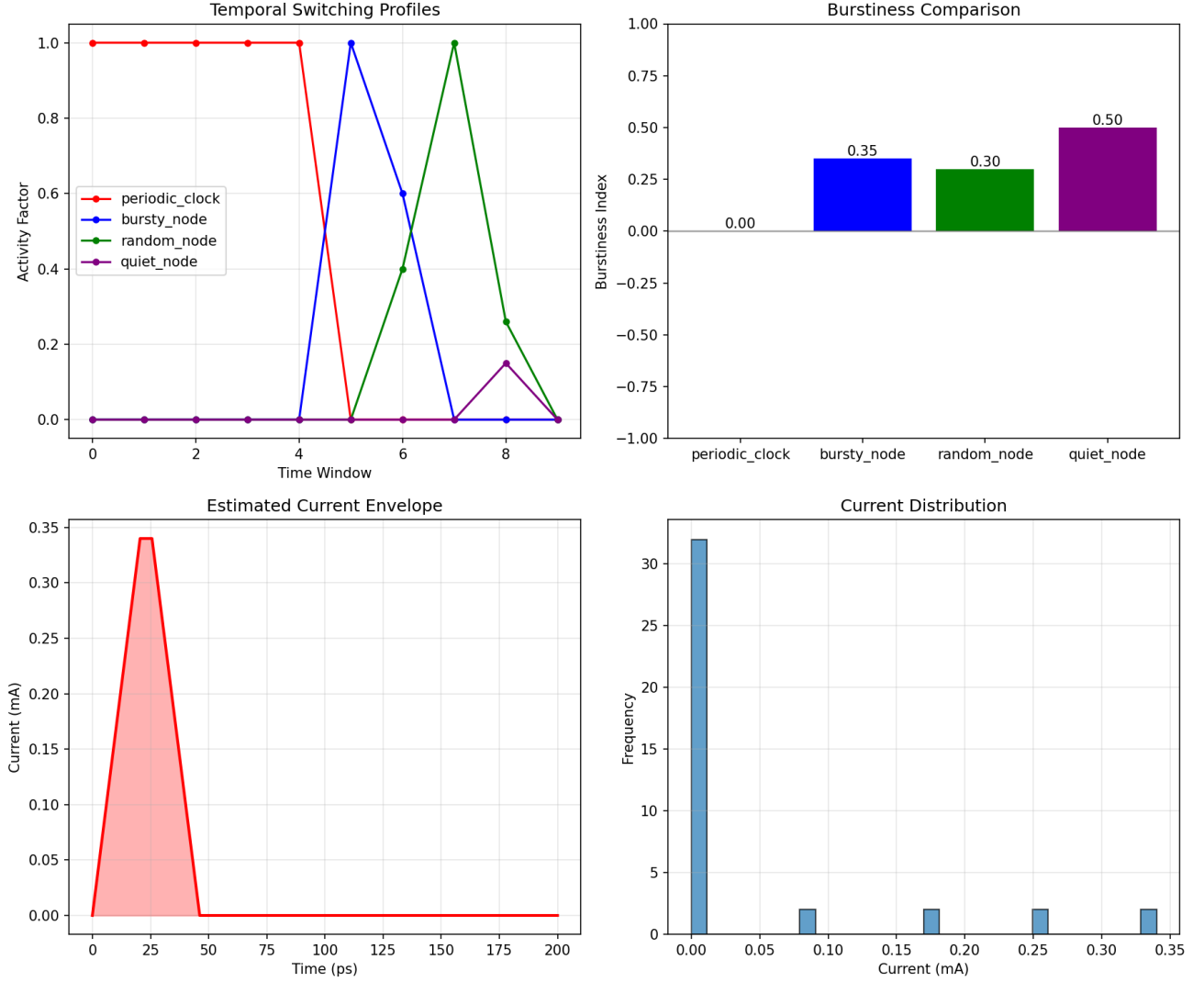


Figure 2: Temporal switching analysis: windowed burstiness for four behavioral regimes and the peak-to-average current envelope (peak 0.34 mA, average 0.04 mA) computed from sketch window counters.

Table VIII: Memory–accuracy frontier on a 10,000-node power-law workload (119,266 events). Global error collapses quickly with memory, but the tail — and therefore per-node power — needs the larger sketches.

sketch size	counters	memory	mean error	hot-node error
Tiny	300	1.2 KB	26,538%	323%
Small	2,000	7.8 KB	3,380%	41%
Medium	10,000	39 KB	349%	5.8%
Large	60,000	234 KB	0.16%	0.01%

IX. WHY SKETCHES MATTER FOR AI-ERA CHIP DESIGN

Three trends make fixed-size, error-bounded summaries more, not less, relevant to chip design.

1. **Scale of the data.** A modern design has millions of gates, and simulation sweeps billions of cycles [1]; AI-driven placement and analysis loop over such data continuously [12]. An $O(n)$ analysis table is the part of that loop that grows without bound — the sketches are the part that does not.
2. **Scale of the questions.** Exploratory analysis mul-

tiplies queries (what-if floorplans, corner sweeps, coverage regressions) far faster than it multiplies stored events. Sketches are query-cheap: estimating a new key costs a lookup, not a rescan, and merging summaries across runs makes incremental answers nearly free.

3. **Scale of the models.** ML models in the loop consume *features* — activity histograms, hot-spot rankings, coverage cardinalities — not raw event logs. Sketches are feature extractors with an explicit error bar, which is what a downstream model can actually use: a noisy-but-bounded feature can be calibrated,

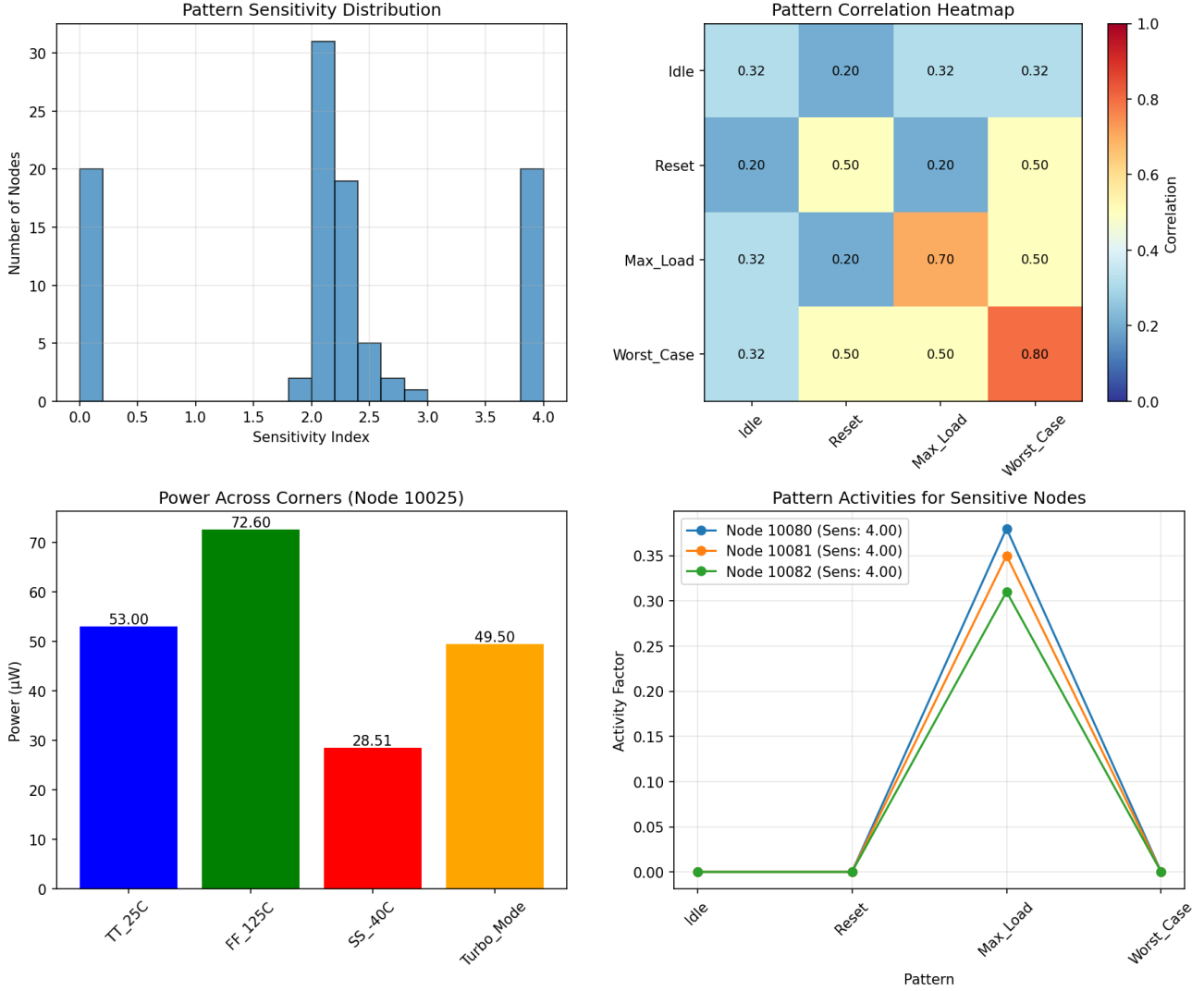


Figure 3: Pattern-dependent and multi-corner switching analysis: per-pattern activity counts with sensitivity index (peak SI = 4.0 at Max_Load) and per-corner power with an 86.6% best-to-worst spread.

an unbounded table cannot.

In short, the sketches are the memory-bounded sensing layer between an unbounded design stream and whatever consumes it — human, script, or model.

X. THREATS TO VALIDITY AND LIMITATIONS

Synthetic workloads and scale. The experiments use generated designs — 500 and 10,000 nodes — rather than industrial netlists, and the largest EDA stream is about 1.2×10^5 events, well below the multi-million-gate, 10^9 -cycle flows that motivate the paper. The *storage* half of the argument does not rest on this scale: the sketch footprint is $O(\varepsilon^{-1} \log \delta^{-1})$ counters, independent of both the design size and N , so a full industrial flow fits the same budget by construction. What small synthetic workloads leave untested is the *error* half — real netlists add spatial-temporal correlation and bus-switching synchronization that a synthetic power-law key distribution

does not reproduce and that can cluster hashes. Heavy-tailed key distributions are precisely where collision-heavy sketches degrade (Section VII), so deployment should re-run the memory-accuracy sweep on representative traces before fixing a budget.

One-sided error and tail queries. Count-Min Sketch never under-counts, so per-node activity is systematically inflated by collision mass; power and thermal budgets derived from it are *pessimistic* by construction — they over-design distribution networks but never under-provision them. Analyses that require conservative lower bounds or unbiased expectations must subtract a bound or use a different structure. The additive guarantee Eq. 1 is absolute, so metrics queried well below εN are not meaningfully answered at that size: the sketch is a poor instrument for *rare-event* questions — dormant-block leakage, isolated timing hazards, rarely triggered trojans — where a low-frequency signature is exactly the signal of interest. Such tasks call for exact or invertible structures sized to the

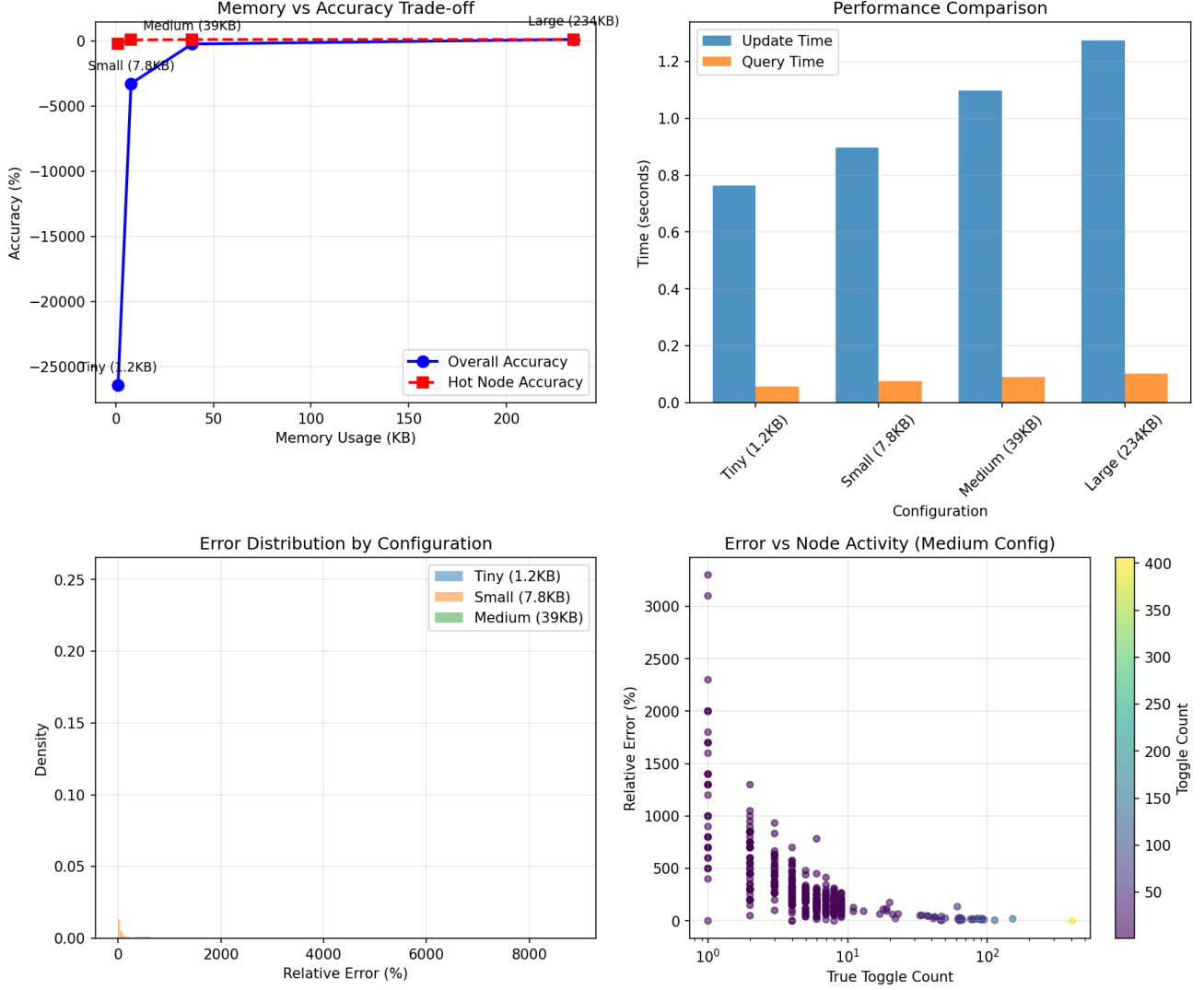


Figure 4: Memory–accuracy trade-off on the 10,000-node power-law workload: mean and hot-node error versus sketch size. Ranking-level accuracy arrives near 39 KB; the distribution tail needs the full 234 KB.

number of distinct rare signatures (e.g. a Bloomier filter [13]), not for a frequency sketch.

Deterministic hashing. Seeding hash parameters from the sketch dimensions makes runs reproducible and merges correct, but also means two adversarially similar key sets always collide the same way; the classical independence assumption behind Eq. 1 is statistical over the key population, not cryptographic.

Hardware dependence. GPU throughput and speed-up depend on the device and are reported at runtime rather than asserted; accuracy, being a property of the register state, does not depend on the device.

Data movement and baselines. The GPU results are on-device throughput (`cpu_time` / `gpu_time`), not end-to-end time: in a complete flow the events must cross PCIe, and the sketch pays off only if on-device compaction saves more transfer than serializing the summary costs. We also

benchmark against no production HDL simulator, so the net system-level speed-up *inside* a real simulation loop is not established here; these experiments show the accuracy and the kernel-level parallelizability, not the wall-clock win in a deployment.

Scope. We treat frequency and cardinality counting only. The counting problems we *do not* cover map to other synopses: delay-distribution percentiles to quantile sketches such as KLL [14] or t-digest [15]; heavy hitters under deletions to Misra–Gries [6] or Space-Saving [16]; rare signatures to exact or invertible structures [13]; and join sizes across hierarchical modules to the broader synopsis family [4]. None are evaluated here.

XI. CONCLUSION

We treated EDA analysis outputs — toggles, violations, patterns, coverage states — as streams and asked what fixed-size summaries can answer. The Count-Min Sketch supplies frequency estimates whose error is additive and

Table IX: Estimates versus ground truth across twelve EDA counting metrics, each from a single-pass HyperLogLog sketch (v0.2.0 run). All errors stay within the $\sigma \approx 1.04/\sqrt{m}$ envelope.

metric	estimate	ground truth	absolute error	relative error
DRC patterns	300,877	290,919	9,958	3.4%
signal paths	498,998	500,000	1,002	0.2%
fanout degrees	49	49	0	0.0%
cell types / region	602	600	2	0.3%
timing paths	99,503	100,000	497	0.5%
critical paths	49,856	50,584	728	1.4%
activity patterns	9,888	9,964	76	0.8%
transition counts	63	63	0	0.0%
states covered	256	256	0	0.0%
transitions covered	65,831	65,506	325	0.5%
defect patterns	138	140	2	1.4%
failing-die buckets	636	637	1	0.2%

quantifiable, instantiated with deterministic, mergeable hashing; HyperLogLog supplies cardinality estimates at $1.04/\sqrt{m}$ error from m bytes, with seven EDA queries built on one register primitive. On twelve EDA metrics the estimates stay within 3.4% of ground truth in a single pass, the 53.1KB switching-power study reproduces the expected clock-dominant power breakdown and an $8.5\times$ current peak-to-average ratio from 64,106 events, and the memory-accuracy sweep makes the failure mode explicit: rank-level answers at 39KB, tail answers at 234KB. Race-free GPU kernels driven by hardware atomics move the hashing and register updates onto the accelerator at no accuracy cost.

The design lesson is a budgeting one: decide the smallest count the analysis will act on, set ε and δ from that decision, and the memory follows — bounded, known before the run, and independent of how long the simulation runs.

XII. CODE AVAILABILITY

All experiments are reproducible from the accompanying open-source package. The Count-Min Sketch and HyperLogLog implementations are in `big_data.basic_count_min_sketch` and `big_data.hyper_loglog_demo`; the case-study pipelines are `big_data.switching_analysis_demo`, `big_data.temporal_switching_analyzer_demo`, `big_data.pattern_dependent_analyzer_demo`, and `big_data.memory_accuracy_tradeoff_demo`; the GPU kernels are `big_data.simple_gpu_demo`, `big_data.better_gpu_demo`, and `big_data.hyper_loglog_gpu_demo`. Each module runs standalone as `python -m big_data.<module>`, and the figures in this paper are generated by those modules.

References

- [1] N. H. E. Weste and D. M. Harris, *CMOS VLSI design: A circuits and systems perspective*, 4th ed. Addison-Wesley, 2011.
- [2] G. Cormode and S. Muthukrishnan, “An improved data stream summary: The Count-Min sketch and its applications,” *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005, doi: [10.1016/j.jalgor.2003.12.001](https://doi.org/10.1016/j.jalgor.2003.12.001).
- [3] P. Flajolet, É. Fusy, C. Goujeon, and É. Marchand-Messelle, “HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm,” in *Analysis of algorithms (AOFA)*, 2007, pp. 127–146.
- [4] G. Cormode, M. Garofalakis, P. J. Haas, and C. Jermaine, “Synopses for massive data: Samples, histograms, quantiles, and heavy hitters,” *Communications of the ACM*, vol. 61, no. 7, pp. 70–77, 2018, doi: [10.1145/3198441](https://doi.org/10.1145/3198441).
- [5] C. Estan and G. Varghese, “New directions in traffic measurement and accounting,” in *ACM SIGCOMM*, 2002, pp. 323–333. doi: [10.1145/633025.633059](https://doi.org/10.1145/633025.633059).
- [6] J. Misra and D. Gries, “Finding repeated elements,” *Science of Computer Programming*, vol. 20, no. 1–2, pp. 145–153, 1993, doi: [10.1016/0167-6423\(93\)90031-9](https://doi.org/10.1016/0167-6423(93)90031-9).
- [7] G. S. Manku and R. Motwani, “Approximate frequency counts over data streams,” in *International conference on very large data bases (VLDB)*, 2002, pp. 346–357.
- [8] N. Alon, Y. Matias, and M. Szegedy, “The space complexity of approximating the frequency moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999, doi: [10.1006/jcss.1998.1601](https://doi.org/10.1006/jcss.1998.1601).
- [9] K.-Y. Whang, B. T. Vander-Zanden, and H. M. Taylor, “A linear-time probabilistic counting algorithm for database applications,” *ACM Transactions on Database Systems*, vol. 15, no. 2, pp. 208–229, 1990, doi: [10.1145/78922.78925](https://doi.org/10.1145/78922.78925).
- [10] E. Durand and P. Flajolet, “LogLog: Deduplication with precise count,” in *European symposium on algorithms (ESA)*, in Lecture notes in computer science, vol. 2832. Springer, 2003, pp. 605–617.
- [11] S. Heule, M. Nunkesser, and A. Hall, “HyperLogLog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm,” in *International conference on extending database technology (EDBT)*, 2013, pp. 683–692.
- [12] A. Mirhoseini, A. Goldie, A. Yazdanbakhsh, et al., “A graph placement methodology for fast chip design,” *Nature*, vol. 594, pp. 207–212, 2021, doi: [10.1038/s41586-021-03544-w](https://doi.org/10.1038/s41586-021-03544-w).

- [13] B. Chazelle, J. Kilian, R. Rubinfeld, and A. Tal, “The Bloomier filter: An efficient data structure for static support lookup tables,” in *ACM-SIAM symposium on discrete algorithms (SODA)*, 2004, pp. 30–39.
- [14] Z. Karnin, K. Lang, and E. Liberty, “Optimal quantile approximation in streams,” in *IEEE symposium on foundations of computer science (FOCS)*, 2016, pp. 71–78. doi: [10.1109/FOCS.2016.17](https://doi.org/10.1109/FOCS.2016.17).
- [15] T. Dunning and O. Ertl, “Computing extremely accurate quantiles using t -digests.” 2019. Available: <https://arxiv.org/abs/1902.04080>
- [16] A. Metwally, D. Agrawal, and A. El Abbadi, “Efficient computation of frequent and top- k elements in data streams,” in *International conference on database theory (ICDT)*, in Lecture notes in computer science, vol. 3363. Springer, 2005, pp. 398–412.